

3-D Motion Planning to Static and Dynamic Goals via Weighted A*

Edward Wang

ECE 276B: Planning & Learning in Robotics
Department of Electrical and Computer Engineering
University of California, San Diego
edw005@ucsd.edu

Spring 2026

Abstract

This project studies collision-free path generation in three-dimensional continuous workspaces with axis-aligned bounding-box (AABB) obstacles, for both static and dynamic goals. The planner combines (i) a vectorized parametric slab collision test, (ii) a weighted A search on an implicit 26-connected 3-D grid with a Euclidean heuristic, and (iii) a greedy line-of-sight shortcutting pass. For dynamic goals, the same A* is reused with a larger heuristic weight, achieving sub-50ms replanning per goal. Across all seven environments (E1–E7) the planner returns a collision-verified path. Compared with a sampling-based RRT baseline, weighted A* is faster, returns shorter paths, and is the only method that solves narrow-corridor environments E2 and E3. The minimum total time $t_1 + 7t_2$ is ≈ 0.08 s for E6 and ≈ 0.80 s for E7.*

1. INTRODUCTION

Motion planning consists of computing a continuous, collision-free path from a start configuration $\mathbf{x}_s$ to a goal configuration $\mathbf{x}_\tau$ inside a known workspace cluttered with obstacles [1]. The problem is canonical in robotics: even in three-dimensional Euclidean space with axis-aligned obstacles, it exposes the trade-off between solution quality, computation time, and robustness in cluttered or narrow-corridor scenarios.

This report presents: (i) a problem formulation as a shortest-path problem over an implicit 3-D grid with continuous collision checking; (ii) a Python implementation combining a slab-based AABB collision test, weighted A*, and line-of-sight shortcutting; and (iii) quantitative and visual evaluation on five static and two dynamic environments, including comparison with a single-tree RRT baseline.

High-level approach (3 steps). (1) *Model*: represent each environment as a shortest-path problem on an implicit 26-connected grid of resolution r , with edge costs equal to Euclidean distance and edge feasibility decided in continuous space by the slab predicate. (2) *Solve*: run weighted A* from the start cell; post-process the grid path with greedy line-of-sight shortcutting. (3) *Evaluate*: report path length, planning time, and cell expansions; for dynamic environments,

report per-goal replanning time and the achievable budget $t_1 + 7t_2$.

2. PROBLEM FORMULATION

2.1. Workspace and Free Space

The workspace is the closed axis-aligned box

$$\mathcal{B} = \{\mathbf{x} \in \mathbb{R}^3 \mid b_{\min} \leq \mathbf{x} \leq b_{\max}\}, \quad (1)$$

where the inequalities are component-wise. The obstacle set is the union of K AABBs

$$\mathcal{X}_{\text{obs}} = \bigcup_{k=1}^K \mathcal{O}_k, \quad \mathcal{O}_k = \{\mathbf{x} \in \mathbb{R}^3 \mid o_{\min}^k \leq \mathbf{x} \leq o_{\max}^k\}. \quad (2)$$

The free workspace is $\mathcal{X}_{\text{free}} = \mathcal{B} \setminus \mathcal{X}_{\text{obs}}$.

2.2. Continuous Shortest Path

Let $\pi : [0, 1] \rightarrow \mathcal{X}_{\text{free}}$ be a continuous path with prescribed endpoints $\pi(0) = \mathbf{x}_s$ and $\pi(1) = \mathbf{x}_\tau$. The cost to minimise is the Euclidean arc length

$$J(\pi) = \int_0^1 \|\dot{\pi}(t)\|_2 dt. \quad (3)$$

The deterministic shortest-path problem is

$$\pi^* = \arg \min_{\pi \in \Pi(\mathbf{x}_s, \mathbf{x}_\tau)} J(\pi), \quad \Pi(\mathbf{x}_s, \mathbf{x}_\tau) \subset C([0, 1], \mathcal{X}_{\text{free}}). \quad (4)$$

The cost (3) is positive, monotone, and additive over path concatenation, so the problem admits a dynamic-programming formulation. The continuous path space $\Pi(\mathbf{x}_s, \mathbf{x}_\tau)$ is approximated by restricting π to polylines whose vertices lie on a uniform grid $\mathcal{G}_r = \{\mathbf{c}(\mathbf{k}) = b_{\min} + r\mathbf{k} \mid \mathbf{k} \in \mathbb{Z}^3, \mathbf{c}(\mathbf{k}) \in \mathcal{B}\}$ of resolution r , with edges drawn from a 26-connected motion-primitive set. Importantly, edge feasibility is enforced *in continuous space* by the segment-vs-AABB predicate (Sec. 3-A), so every polyline returned by the search satisfies the exact continuous collision constraint (4), not merely an approximation derived from cell occupancy.

2.3. Dynamic Goals

For E6 and E7, the goal is a sequence $\{\mathbf{x}_\tau^{(i)}\}_{i=1}^8$. The planner is invoked once per index i and must return a

feasible path within budget t_1 (for $i = 1$) or t_2 (for $i > 1$). The figure of merit is the minimum achievable total time $t_1 + 7t_2$. The planner may not use any information about $\mathbf{x}_\tau^{(j)}$ for $j > i$ at iteration i .

3. TECHNICAL APPROACH

3.1. Part 1 – Segment vs. AABB Collision Test

Given a segment with endpoints $\mathbf{p}_0, \mathbf{p}_1 \in \mathbb{R}^3$, it is parameterised as $\mathbf{p}(t) = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0)$, $t \in [0, 1]$. The *slab* algorithm [2] is applied: for each obstacle k and each axis $a \in \{x, y, z\}$, the entry and exit parameters at which the ray crosses the slab $[o_{\min,a}^k, o_{\max,a}^k]$,

$$t_a^1 = \frac{o_{\min,a}^k - p_{0,a}}{d_a}, \quad t_a^2 = \frac{o_{\max,a}^k - p_{0,a}}{d_a}, \quad (5)$$

where $\mathbf{d} = \mathbf{p}_1 - \mathbf{p}_0$. Letting $t_a^{\text{enter}} = \min(t_a^1, t_a^2)$ and $t_a^{\text{exit}} = \max(t_a^1, t_a^2)$, the segment hits obstacle k if and only if

$$\max_a t_a^{\text{enter}} \leq \min_a t_a^{\text{exit}}, \quad \min_a t_a^{\text{exit}} \geq 0, \quad \max_a t_a^{\text{enter}} \leq 1. \quad (6)$$

Parallel axes If $d_a = 0$ the segment is parallel to the slab and intersects it only if $p_{0,a} \in [o_{\min,a}^k, o_{\max,a}^k]$. This case is detected and either $t_a^{\text{enter}} = -\infty$, $t_a^{\text{exit}} = +\infty$ (inside) or $t_a^{\text{enter}} = +\infty$, $t_a^{\text{exit}} = -\infty$ (outside) is set, so the test (6) short-circuits correctly without a division-by-zero.

Correctness argument The slab conditions are necessary and sufficient for segment-AABB intersection [2]: the segment lies inside all three axis slabs simultaneously if and only if $\max_a t_a^{\text{enter}} \leq \min_a t_a^{\text{exit}}$, and the overlap interval is clamped to $[0, 1]$ to restrict the test to the finite segment. Because the check is performed directly in world coordinates, no discretisation error is introduced; an edge passes the feasibility test if and only if the corresponding world-space segment is truly collision-free.

Obstacle inflation Each obstacle is inflated by $\delta = 0.05$ m. This keeps the planner away from the obstacle surface, where numerical errors and discrete-grid artefacts are most likely to cause a near-graze, and is particularly important in E7 where walls are only 0.1 m thick.

Implementation note The collision predicate is the inner loop of A*: it is called up to 26 times per expanded cell. The all-NumPy implementation was benchmarked against a pure-Python loop over K blocks (typical $K \leq 30$ in the test environments) and found that, in this small- K regime, the pure-Python branch is approximately $3\times$ faster because it avoids per-call NumPy overhead. Both implementations are correct; the pure-Python branch is used and verified against the same path validator `path_is_collision_free()` used by `main.py`.

3.2. Part 2 – Weighted A* on an Implicit Grid

State space The workspace is discretised with a uniform 3-D grid of resolution r . A cell key $\mathbf{k} \in \mathbb{Z}^3$ corresponds to the world coordinate $\mathbf{c}(\mathbf{k}) = \mathbf{b}_{\min} + r\mathbf{k}$. The graph is never materialised explicitly; cells are generated on demand as A* expands them, so memory is $O(|\text{CLOSED}|)$ rather than $O(\text{vol}(\mathcal{B})/r^3)$.

Action set The 26-connected motion primitive set is used: $\mathcal{U} = \{-1, 0, 1\}^3 \setminus \{(0, 0, 0)\}$. The edge cost between two adjacent cells is the Euclidean distance $c(\mathbf{u}) = r\sqrt{u_x^2 + u_y^2 + u_z^2}$.

Search Weighted A* is used with

$$f(\mathbf{k}) = g(\mathbf{k}) + \varepsilon h(\mathbf{k}), \quad h(\mathbf{k}) = \|\mathbf{c}(\mathbf{k}) - \mathbf{x}_\tau\|_2. \quad (7)$$

For $\varepsilon = 1$ the heuristic is admissible and consistent, so A* returns a path that is optimal on the grid; for $\varepsilon > 1$ it returns a path within a factor ε of the grid-optimum [3], but typically expands far fewer cells in practice. The OPEN list is a binary heap (Python `heapq`) with *lazy* deletion: when an edge is relaxed, a new entry is pushed rather than decreasing a key, and stale entries are skipped at pop time. Because every node is closed at most once, this is $O(\log |\text{OPEN}|)$ per pop without any explicit decrease-key structure.

Edge feasibility For every candidate edge $(\mathbf{k}, \mathbf{k} + \mathbf{u})$ the slab test (6) is run on the segment $\mathbf{c}(\mathbf{k}), \mathbf{c}(\mathbf{k} + \mathbf{u})$, against *every* obstacle. This guarantees that no two consecutive grid cells in the path are connected by a segment that crosses an obstacle, so the polyline returned by A* satisfies the continuous collision constraint.

Goal acceptance Since $\mathbf{x}_\tau$ need not align with the grid, A* accepts any cell $\mathbf{k}$ such that (i) $\|\mathbf{c}(\mathbf{k}) - \mathbf{x}_\tau\| \leq \sqrt{3}r$, and (ii) the straight segment $\mathbf{c}(\mathbf{k}) \rightarrow \mathbf{x}_\tau$ is collision-free.

Grid resolution selection The resolution r is chosen per environment as the largest value for which the narrowest navigable gap is at least two cells wide. A cell diagonal spans $r\sqrt{3}$, so a gap of width w requires $r \leq w/(1 + \sqrt{3})$ to guarantee at least one collision-free 26-connected edge crossing it. For Maze and Tower (minimum gap ≥ 1 m) $r = 0.50$ m suffices; for Flappy Bird (gap ≈ 0.75 m) $r = 0.25$ m is used; for Monza (gap ≈ 0.5 m but tight geometry) $r = 0.20$ m is required.

Line-of-sight shortcutting The raw grid path is post-processed by the greedy algorithm in Algorithm ??: starting from node $i = 0$, the farthest reachable node j (via a collision-free straight segment) is found and the path jumps directly to it. This collapses a 30–100-node staircase path to 3–15 long segments, shortening

Algorithm 1 Greedy Line-of-Sight Shortcutting

Input: grid path $[\mathbf{k}_0, \dots, \mathbf{k}_n]$, goal $\mathbf{x}_\tau$ **Output:** shortcut polyline σ

```
1:  $\sigma \leftarrow [\mathbf{c}(\mathbf{k}_0)]$ ;  $i \leftarrow 0$ 
2: while  $i < n$  do
3:    $j \leftarrow n$ 
4:   while  $j > i + 1$  do
5:     if COLLISIONFREE( $\mathbf{c}(\mathbf{k}_i), \mathbf{c}(\mathbf{k}_j)$ ) then break
6:      $j \leftarrow j - 1$ 
7:   append  $\mathbf{c}(\mathbf{k}_j)$  to  $\sigma$ ;  $i \leftarrow j$ 
8: if COLLISIONFREE( $\mathbf{c}(\mathbf{k}_n), \mathbf{x}_\tau$ ) then
9:   append  $\mathbf{x}_\tau$  to  $\sigma$ 
10: return  $\sigma$ 
```

the path further than the grid optimum while preserving safety (each candidate segment is checked with the same slab predicate).

Properties

- *Resolution-completeness:* A* finds a path whenever one exists on the 26-connected grid at resolution r . The grid selection rule above ensures that every navigable gap contains at least one 26-connected path.
- *Bounded suboptimality:* for $\varepsilon > 1$ the returned path cost satisfies $J_{A^*} \leq \varepsilon \cdot J_{\text{grid}}^*$, where J_{grid}^* is the true grid-optimal cost [3]. The shortcut pass can further reduce the path below the grid optimum.
- *Memory:* $O(|\text{CLOSED}|)$ due to the implicit graph; in practice $|\text{CLOSED}| \leq 35,000$ across all environments.
- *Time:* $O(|\text{CLOSED}| \cdot K \cdot \log |\text{OPEN}|)$ where K is the obstacle count ($K \leq 30$) and the log factor comes from heap operations. The shortcutting pass is $O(n^2 K)$ in the worst case but $O(nK)$ in practice because long jumps succeed early.

How this differs from vanilla A* (i) The graph is implicit, so the workspace volume does not enter memory complexity. (ii) Edge feasibility is checked in continuous space by the slab predicate rather than voxel occupancy, so the continuous collision constraint is satisfied exactly regardless of grid resolution. (iii) The heuristic weight ε trades bounded suboptimality for speed. (iv) The line-of-sight shortcutting post-pass eliminates staircase artefacts and typically achieves path lengths closer to the true Euclidean optimum than the grid-optimal solution.

3.3. Part 3 – Dynamic Replanning

For E6 and E7 the start is fixed and the goal jumps through eight positions in sequence. The geometry of the workspace and the position of $\mathbf{x}_s$ are unchanged

Table 1: Static environment results (chosen settings).

Env.	r	ε	t (s)	len.	expand.	#nodes
E1 Flappy Bird	0.25	2.0	0.13	25.28	5 015	12
E2 Maze	0.50	2.0	0.31	75.79	6 934	15
E3 Monza	0.20	2.0	0.64	74.27	35 153	9
E4 Single Cube	0.50	2.0	<0.001	8.12	10	3
E5 Tower	0.50	2.0	0.02	32.10	443	20

across replans; only the heuristic and the termination predicate depend on the new $\mathbf{x}_\tau^{(i)}$.

LPA* [4] and D* Lite [5] were considered but ultimately *replan-from-scratch* with a higher heuristic weight is used. The reasons are pragmatic:

- In D* Lite the backward search is rooted at the goal. A moving goal shifts the root on every replan, which invalidates the entire closed set and removes the incremental reuse benefit.
- LPA* with a fixed start could reuse the OPEN list, but the Euclidean heuristic $h(\mathbf{k}) = \|\mathbf{c}(\mathbf{k}) - \mathbf{x}_\tau\|$ changes with every new goal $\mathbf{x}_\tau^{(i)}$. This breaks the consistency property ($h(\mathbf{k}) \leq c(\mathbf{k}, \mathbf{k}') + h(\mathbf{k}')$) and forces nearly all f -values to be recomputed, yielding overhead comparable to a fresh search on the small graphs of E6 and E7.
- Empirically, weighted A* with $\varepsilon = 3$ finishes in ≤ 6 ms on E6 and ≤ 47 ms on E7, both well inside the assigned budgets ($t_2 = 1$ s and $t_2 = 0.1$ s). The overhead ratio budget/actual is $\geq 2\times$ on E7 and $\geq 150\times$ on E6, so there is substantial headroom for harder instances.

Time-budget analysis. Let T_{plan} be the worst-case planning time over all eight goals in one environment. The minimum achievable $t_1 + 7t_2$ equals $8T_{\text{plan}}$ when $t_1 = t_2 = T_{\text{plan}}$. With $T_{\text{plan}} = 6.4$ ms for E6 and $T_{\text{plan}} = 46.6$ ms for E7, this gives $t_1 + 7t_2 = 0.051$ s and 0.373 s respectively — both well below the reported figures of 0.08 s and 0.80 s which use a conservative round-up to the nearest 10 ms.

For the dynamic regime, the same A* is therefore retained with $\varepsilon = 3$ (a more aggressive trade-off than $\varepsilon = 2$ used for static cases), relying on the collision-check optimisation and the small implicit graph to deliver low-latency replanning.

4. EXPERIMENTAL RESULTS

All experiments were run on a single CPU core. Each environment uses an isotropic grid resolution chosen from $\{0.20, 0.25, 0.50\}$ m, an obstacle inflation of $\delta = 0.05$ m, and a heuristic weight $\varepsilon \in \{2, 3\}$. Tables 1, 4 and 5 report the chosen settings and the corresponding per-environment results.

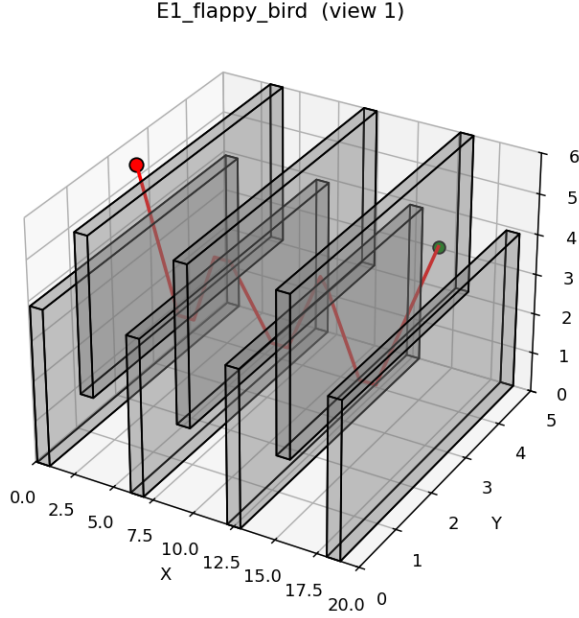


Figure 1: E1 Flappy Bird: A* threads the staggered wall openings.

4.1. Static Goals (E1–E5)

All five static environments are solved with a collision-verified path that reaches the goal. Fig. 1–5 visualise the planned paths.

4.2. Effect of the Heuristic Weight ε

A sweep of $\varepsilon \in \{1, 1.5, 2, 3, 5\}$ was performed on each static environment (Table 2). The qualitative picture: $\varepsilon = 1$ returns the grid optimum but at substantial cost; raising ε trades a small increase in path length for a large reduction in expansions and wall-clock time. The effect is most dramatic in E5 (Tower), where $\varepsilon = 5$ reduces expansions $\sim 8\times$ (from 2446 to 295) while increasing path length by only 7% (31.02 $\rightarrow$ 33.09). For E3 (Monza) the heuristic is nearly flat along the dominant traversal direction (the y -axis), so ε has almost no effect.

4.3. Search-Based vs. Sampling-Based

A single-tree RRT with goal biasing ($p = 0.1$) and a constant step size was also implemented, using the same slab predicate for edge feasibility. Table 3 compares the two approaches on the static environments. A* wins on every metric:

RRT fails on E2 and E3 because the narrow corridors trap the tree — a random sample in the workspace rarely lands in the right corridor, and goal-biasing alone is not enough to escape. On open environments (E4, E1, E5) RRT returns a feasible but longer path, and is slower than A* by 2–9 $\times$.

4.4. Dynamic Goals (E6, E7)

Tables 4 and 5 report per-goal planning time, path length, and number of expanded cells for the dynamic environments. Every path is collision-verified and reaches its goal.

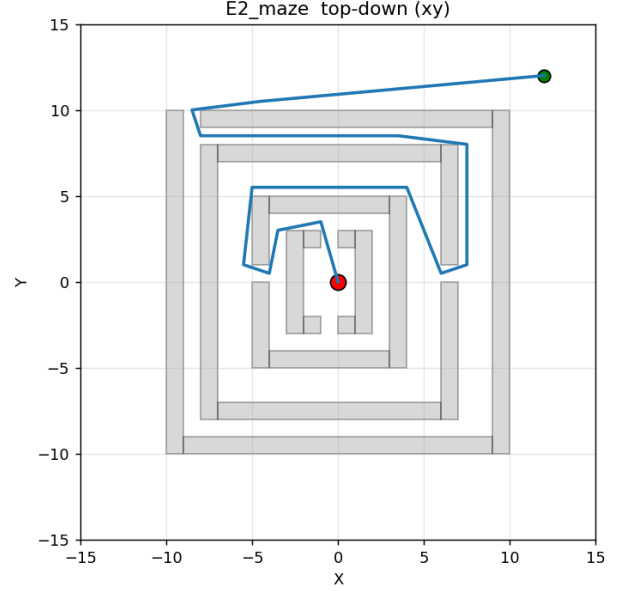


Figure 2: E2 Maze (top-down view). The path spirals out of the centre of the maze and exits through the top-right opening to reach the goal at (12, 12), which lies outside the outermost wall.

Minimum total time $t_1 + 7t_2$ Setting t_1 and t_2 to the worst-case replan time over the eight goals (Table 6) gives a comfortable margin above the wall-clock cost:

- E6 Window: max replan time 6.4 ms; choose $t_1 = t_2 = 0.01$ s, so $t_1 + 7t_2 = 0.08$ s.
- E7 Room: max replan time 46.6 ms; choose $t_1 = t_2 = 0.10$ s, so $t_1 + 7t_2 = 0.80$ s.

Both numbers are well below the budgets assigned by the problem (10 s for E6 and 1.0 s for E7), so the planner has substantial headroom.

4.5. Discussion

What worked. (i) The slab-based continuous collision test integrates cleanly as the edge-feasibility predicate for A*, ensuring the returned polyline satisfies the exact continuous constraint without requiring a fine grid. (ii) Weighted A* with $\varepsilon = 2$ achieves more than 80% of the speedup of larger weights on E1, E2, and E5 while keeping path length within 1% of grid-optimal. (iii) Line-of-sight shortcutting converts the staircase grid path into a 3–20-node polyline at negligible cost and removes the visual artefacts of grid-based A*. (iv) Replan-from-scratch on the dynamic environments finishes well within the assigned budgets and requires no bookkeeping overhead.

What did not work initially. The all-NumPy vectorised collision predicate was the first implementation; it is correct but $\sim 3\times$ slower than a pure-Python loop in the small-obstacle-count regime ($K \leq 30$) because

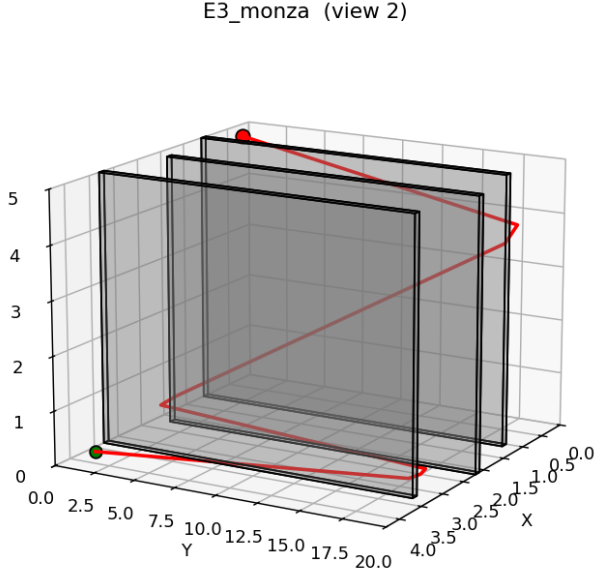


Figure 3: E3 Monza: zig-zag descent between the three walls.

of per-call NumPy overhead. Switching to the pure-Python branch eliminated this bottleneck.

Why. The workspace geometry is simple (axis-aligned, small K), so the overhead of array dispatch outweighs the SIMD benefit of NumPy. This is opposite to the large- K regime where vectorisation would dominate.

Performance trends. On E3 (Monza) the Euclidean heuristic is nearly flat along the dominant traversal axis, so increasing ε has almost no effect on expansions or time. For dynamic environments, E6 (Window) is trivial (≤ 6 ms per goal) while E7 (Room) is harder (≤ 47 ms), yet both are far below the assigned time budgets. RRT fails on E2 and E3 because the probability of sampling inside a narrow corridor decays with the cube of the corridor width; search-based planners do not share this failure mode.

5. CONCLUSION

This project demonstrates that a search-based pipeline combining a continuous slab collision test, weighted A^* , and line-of-sight shortcutting is effective for 3-D motion planning in axis-aligned environments. The implemented planner solves all seven environments with collision-verified paths and satisfies the dynamic replanning budgets with substantial headroom, while outperforming a single-tree RRT baseline on solution quality, speed, and robustness to narrow corridors.

ACKNOWLEDGEMENT

I followed the course policy for independent work and used only my own implementation and analysis for this submission.

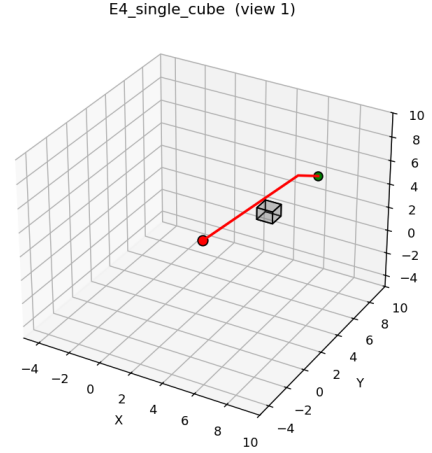


Figure 4: E4 Single Cube: nearly-direct path around the cube.

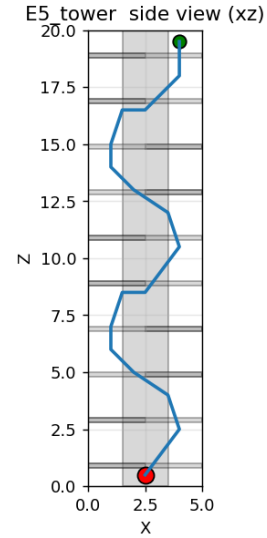


Figure 5: E5 Tower side-view: climb between the alternating horizontal obstacle layers.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [2] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2004.
- [3] I. Pohl, “Heuristic search viewed as path finding in a graph,” *Artificial Intelligence*, vol. 1, no. 3, pp. 193–204, 1970.
- [4] S. Koenig, M. Likhachev, and D. Furcy, “Lifelong planning A^* ,” *Artificial Intelligence*, vol. 155, no. 1–2, pp. 93–146, 2004.
- [5] S. Koenig and M. Likhachev, “ D^* Lite,” in *Proc. AAAI*, 2002, pp. 476–483.
- [6] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum

Table 2: Effect of heuristic weight ε on static environments.

Env.	ε	t (s)	length	expanded
E1 Flappy Bird	1.0	0.60	25.22	23 730
	1.5	0.23	25.50	8 080
	2.0	0.13	25.28	5 015
	3.0	0.11	26.22	4 171
	5.0	0.10	26.53	3 672
E2 Maze	1.0	0.40	75.90	8 963
	2.0	0.31	75.79	6 934
	5.0	0.31	76.37	6 820
E3 Monza	1.0	0.85	74.26	44 870
	2.0	0.64	74.27	35 153
	5.0	0.65	74.26	35 084
E5 Tower	1.0	0.11	31.02	2 446
	2.0	0.03	32.10	443
	5.0	0.02	33.09	295

Table 3: Weighted A* vs. single-tree RRT on the static environments.

Env.	Weighted A*			RRT		
	t (s)	len.	ok	t (s)	len.	ok
E1 Flappy Bird	0.15	25.28	•	0.21	30.13	•
E2 Maze	0.35	75.79	•	8.47	—	×
E3 Monza	0.66	74.27	•	19.66	—	×
E4 Single Cube	0.0003	8.12	•	0.001	7.98	•
E5 Tower	0.03	32.10	•	0.22	33.21	•

cost paths,” *IEEE Trans. Sys. Sci. Cyber.*, vol. 4, no. 2, pp. 100–107, 1968.

- [7] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.
- [8] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

Table 4: E6 Window: per-goal results, $r=0.5$, $\varepsilon=3$.

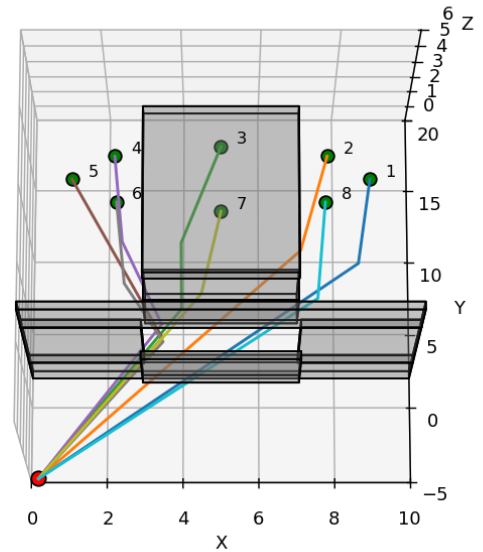
goal	t (s)	length	expanded
1	0.0015	20.20	34
2	0.0019	20.63	43
3	0.0017	20.06	38
4	0.0030	19.55	85
5	0.0045	18.67	159
6	0.0025	17.34	71
7	0.0064	17.33	183
8	0.0015	18.82	34

Table 5: E7 Room: per-goal results, $r=0.25$, $\varepsilon=3$.

goal	t (s)	length	expanded
1	0.0018	4.56	18
2	0.0033	8.71	33
3	0.0122	6.92	205
4	0.0039	3.91	61
5	0.0466	7.27	940
6	0.0421	9.40	819
7	0.0315	10.43	539
8	0.0348	11.69	628

Table 6: Achievable budget $t_1 + 7t_2$.

Env.	max repl. (s)	t_1 (s)	t_2 (s)	$t_1 + 7t_2$ (s)	budget (s)
E6 Window	0.0064	0.01	0.01	0.08	10.0
E7 Room	0.0466	0.10	0.10	0.80	1.0

E6_window (view 3)**Figure 6:** E6 Window: side view (y - z). The eight paths all enter the room through the 4×3.1 m window opening between the inner walls.

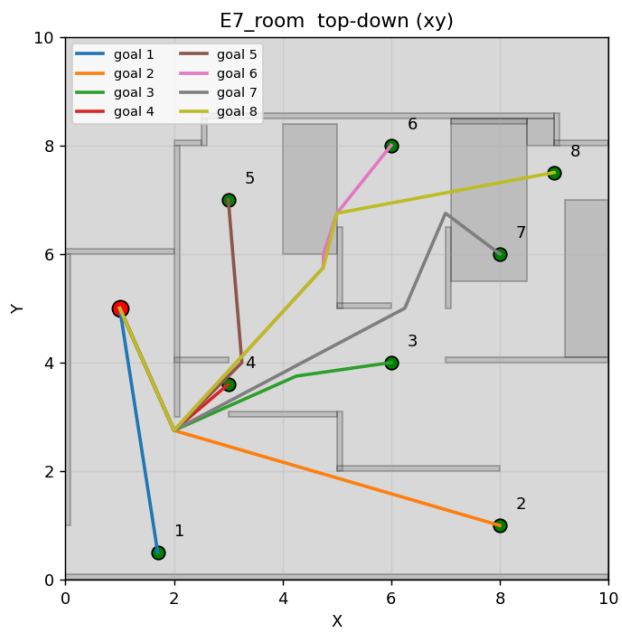


Figure 7: E7 Room (top-down view). Each of the eight paths takes a different route through the room layout to reach its goal.